Spectra

Distributed Spectrum Intelligence at Scale

Simon Knight, Ph.D.

Principal Architect, AutoNetKit Research

March 2026 • Version 2.4.0

Last updated: March 11, 2026

Abstract. Spectra is a distributed spectrum intelligence platform that autonomously monitors, classifies, and analyses radio-frequency signals across the HF–UHF range. By combining edge-accelerated DSP, multi-stage machine learning, and LLM-based RAG synthesis, Spectra reduces gigabytes of raw IQ data into actionable, narrated briefings. This report serves as the definitive technical reference for the system's architecture and design logic.

Contents

Executive Summary

The modern radio-frequency (RF) environment is characterized by extreme entropy. A single 10 MHz wideband capture produces 80 MB/s of raw IQ data, most of which is noise or known "white space." Traditional monitoring tools require human operators to manually identify signals of interest (SOI), a process that does not scale to the density of modern signal environments.

Spectra solves this scaling problem through **Deterministic Entropy Reduction**. It treats the RF spectrum not as a series of samples to be recorded, but as a stream of information to be distilled. The platform implements a hierarchical pipeline that:

1. **Detects**: Identifies energy bursts at the capture edge.
2. **Classifies**: Labels modulations using 1D-ResNet ML models.
3. **Identifies**: Matches emitter fingerprints against known databases.
4. **Decodes**: Extracts protocol-level content (pager messages, radio IDs).
5. **Synthesizes**: Generates natural-language briefings using RAG.

This four-tier, distributed architecture allows for massive horizontal scaling, where low-cost edge sensors (RTL-SDR/AirSpy) feed a central intelligence orchestrator, providing a "God's eye view" of the spectrum without the need for constant human supervision.

List of Design Decisions & Rules

1 Introduction

Spectrum monitoring has transitioned from a manual signal-analysis craft to a computational intelligence discipline.

Insight: *Autonomous spectrum intelligence is not about capturing samples, but about the deterministic reduction of high-bandwidth entropy into low-bandwidth actionable tokens.*

: Latency-Critical Edge DSP

Signal detection and feature extraction must occur at the capture edge to minimize backhaul bandwidth. Only metadata and signal-of-interest (SOI) fragments are permitted to traverse the distribution tier.

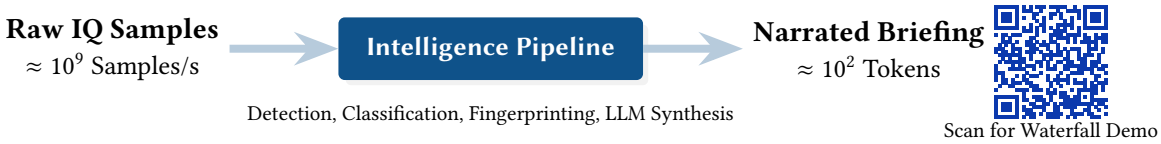

Figure 1. The Spectra Synthesis Funnel and Live Dashboard Link.

1.1 What Spectra Solves

Spectrum monitoring has traditionally required either expensive, proprietary receivers that lock the operator into a single vendor's analysis software, or fragmented open-source tools that each handle one piece of the pipeline: one tool to tune the radio, another to display a waterfall, another to decode a protocol. Stitching these together into an autonomous system that detects, classifies, identifies, and reports on signals around the clock is left as an exercise for the operator.

Spectra replaces this patchwork with a single, distributed platform. A user connects one or more software-defined radios (SDRs), points Spectra at a frequency range, and receives classified, enriched intelligence — modulation labels, protocol decodes, emitter fingerprints, and natural-language briefings — without manual intervention.

1.2 Design Philosophy

Two principles guide every design decision in Spectra:

1. **Edge-to-intelligence pipeline.** Every stage from IQ acquisition to briefing generation is connected by typed, binary protocols. There is no file-based hand-off between stages; data flows as streams. A signal detected at the edge can reach a narrated briefing in under one second.
2. **Thin clients, thick server.** The Rust TUI and React web UI are intentionally thin. They render data and accept user commands. All DSP, ML inference, protocol decoding, and mission logic lives in the Python backend. This means adding a new client (mobile, embedded) requires only implementing the TLV WebSocket protocol, not reimplementing the intelligence pipeline.

1.3 Who Should Read This Document

- **SDR operators** deploying Spectra for spectrum monitoring who want to understand how classification, detection, and missions work under the hood.
- **Developers** extending the platform with new decoders, classifiers, or client applications, needing to understand the module structure and API contracts.
- **Researchers** building on Spectra's synthetic training lab or emitter fingerprinting capabilities.

1.4 How to Read This Document

Section 2 covers the data model — the structures that flow between components. Section 3 presents the four-tier architecture. Sections 7–17 cover the six core subsystems in depth, ordered from deepest (signal detection) to broadest (audio intelligence). Section 25 documents the REST and WebSocket API surface. Section 29 lists known limitations with workarounds.

Readers wanting a quick start should read §3 (Architecture) and §25 (API Reference), then refer to individual subsystem sections as needed.

2 Data Model

Spectra's data model centres on four structures that flow through the pipeline.

2.1 DetectedSignal

The atomic unit of the detection pipeline. Produced by the energy detector and consumed by every downstream stage.

Listing 1. DetectedSignal dataclass

```python
@dataclass
class DetectedSignal:
    freq_start_hz: float   # Lower edge of detected emission
    freq_end_hz: float     # Upper edge
    power_db: float        # Peak power in dB
    mean_power_db: float   # Mean power across bandwidth
    bandwidth_hz: float    # freq_end - freq_start
```

2.2 SignalClassification

The output of the two-stage classification pipeline. Carries the ML label, confidence, alternatives, and optional SigIDWiki enrichment.

Listing 2. SignalClassification dataclass

```python
@dataclass
class SignalClassification:
    label: str                              # e.g. "FM", "QPSK", "POCSAG"
    confidence: float                       # 0.0--1.0 calibrated probability
    alternatives: list[tuple[str, float]]   # Runner-up labels
    freq_start_hz: float
    freq_end_hz: float
    timestamp_s: float
    is_retrospective: bool = False          # True if from deep analysis
    sigidwiki_matches: list[SignalMetadata] = []
    metadata_source: str = "ml"             # "ml", "sigidwiki", "combined"
```

2.3 Signal Hit (Persistent Record)

When a detection is significant enough to persist, it becomes a row in the DuckDB `signal_hits` table:

Listing 3. signal_hits schema

```sql
CREATE TABLE signal_hits (
    id              VARCHAR PRIMARY KEY,
    timestamp_ms    BIGINT NOT NULL,
    center_freq_hz  DOUBLE NOT NULL,
    sample_rate_hz  DOUBLE NOT NULL,
    freq_start_hz   DOUBLE NOT NULL,
    freq_end_hz     DOUBLE NOT NULL,
    bandwidth_hz    DOUBLE NOT NULL,
    peak_db         DOUBLE NOT NULL,
    mean_power_db   DOUBLE NOT NULL,
    device_name     VARCHAR NOT NULL,
    emitter_id      VARCHAR,      -- FK to emitters table (SEI)
    label           VARCHAR,      -- classification label
    confidence      DOUBLE,
    artifact_path   VARCHAR       -- path to SigMF recording
);
```

2.4 Mission Lifecycle

Missions are the unit of autonomous operation. A `MissionDef` describes *what* to scan; a `MissionRun` tracks the execution state.

Listing 4. Mission data model (simplified)

```python
@dataclass
class MissionDef:
    name: str
    freq_start_hz: float
    freq_end_hz: float
    dwell_ms: int
```

```
    modulations: list[str]        # filter to these modulation types
    capture_level: CaptureLevel   # BASIC | DEEP | FULL

@dataclass
class MissionRun:
    run_id: str
    def_id: str
    state: MissionStatus          # queued | running | paused | completed
    created_at: datetime
    events: list[MissionEvent]
```

Capture levels

`BASIC`: classification only. `DEEP`: classification + protocol decode + SigMF recording. `FULL`: all of DEEP plus SEI fingerprinting and emitter vector storage.

3 Architecture

3.1 Four-Tier Design

Source

Spectra is composed of four cooperating tiers, each in its own process:

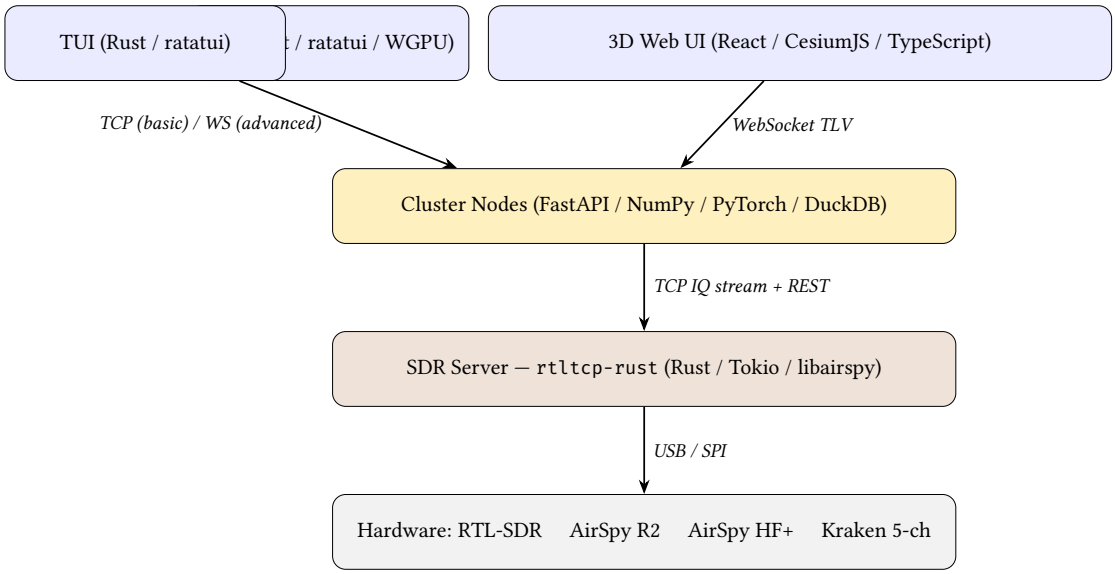

Figure 2. Spectra four-tier architecture. Thin arrows indicate the primary data-flow direction (IQ flows up; commands flow down).

3.2 Technology Stack

Layer	Technology	Purpose
SDR Server	Rust, Tokio, libairspy, rtlsdr-sys	Hardware abstraction, IQ streaming
Backend	Python 3.10+, FastAPI, Uvicorn	Orchestration, DSP, ML, missions
ML Inference	PyTorch, CoreML, ONNX Runtime	Modulation classification
Transcription	mlx-whisper (Apple MLX)	Speech-to-text on Apple Silicon
Persistence	DuckDB	Signal census, emitter database
Vector Search	ChromaDB	Emitter embedding similarity
NL Querying	Ollama (llama3.2:1b)	Natural-language briefings
TUI	Rust, ratatui, crossterm, WGPU	Terminal interface, GPU waterfall
3D Web UI	React, TypeScript, CesiumJS	Browser-based 3D battlespace visualisation

3.3 Project Layout

Path	Contents
`src/spectra/core/`	Orchestrator, DSP, streaming, protocol handlers
`src/spectra/intelligence/`	Classifier, detector, decoders, fingerprinting, missions
`src/spectra/autonomy/`	Autopilot, satellite scheduler, SigMF recorder
`src/spectra/api/`	FastAPI routes (REST + WebSocket)
`crates/spectra-tui/`	Rust TUI application
`signals/spectra-web/`	React web frontend

3.4 Wire Protocol: TLV over WebSocket

All client–server communication uses a custom Type-Length-Value binary protocol over WebSocket. This was chosen over JSON-per-message because spectrum data is high-bandwidth (1000+ FFT frames/sec) and benefits from compact binary encoding.

Listing 5. TLV envelope format

```
# 8-byte envelope: [channel: u8] [flags: u8] [seq: u16 LE] [length: u32 LE]
# Followed by: [payload: N bytes]
HEADER_SIZE = 8

class Channel(IntEnum):
    SPECTRUM       = 0x01  # Binary: u8 bins with 16-byte header
    DETECTIONS     = 0x02  # JSON array of detected signals
    CLASSIFICATION = 0x03  # JSON classification result
    AUDIO          = 0x04  # Binary: i16 LE PCM with header
    CONTROL        = 0x05  # JSON command/response
    META           = 0x06  # JSON session metadata
    SPARSE_IQ      = 0x07  # Binary: complex64 IQ (edge ingest)
    DEMAND_CONTROL = 0x08  # JSON: server->edge demand toggles
```

> **Compression**
>
> Flag bit 0 (`COMPRESSED`) indicates zlib-deflated payload. This is used for large JSON payloads (detections, classifications) but never for spectrum or audio data, where the overhead of compression exceeds the size saving.

4 Intelligence Orchestration & Core DSP

The Python backend acts as the central intelligence hub, mediating between the hardware edge and the client interfaces.

4.1 Core DSP Primitives

All digital signal processing (DSP) operations are decoupled from the UI and networking layers. The core primitives provide:

Source

- **FFT Windowing & Caching**: Pre-allocates Hann/Blackman windows for common FFT sizes (1024, 2048, 4096) to eliminate allocation overhead in the hot path.
- **Rational Resampling**: Combines interpolation and decimation using polyphase FIR filters (via `scipy.signal.resample_poly`), crucial for narrowband audio demodulation.
- **Decimation**: FIR anti-aliasing followed by downsampling to isolate target bandwidths from wideband captures.

4.2 Intelligent Orchestrator

The `IntelligentOrchestrator` binds the pipeline together. It manages:

Source

- **IQ Buffer Lifecycle**: Coordinating the `CircularIQBuffer` and routing chunks to the detection and ML classification engines.

- **Decoder Subprocesses**: Starting and stopping `multimon-ng` and `dsdcc` based on classification hits.
- **Event Pub/Sub**: Decoupling detection events from mission logic using the `DetectionPublisher`.

4.3 Pub/Sub Messaging

To maintain low latency, Spectra utilizes an internal asynchronous Pub/Sub architecture. When the energy detector identifies a burst, it fires a `DetectedSignal` event. Subscribers (like the ML Classifier, the Autopilot Scorer, and the WebSocket Streamer) receive this event independently, preventing slow consumers from blocking the real-time DSP loop.

Source

5 Information Theory: Entropy and Reduction

The fundamental challenge of Spectra is the processing of wideband RF data at scale.

We define the Intelligence Expansion E as the ratio of input bandwidth B_{in} to the semantic token rate R_{out}. For a 20MHz wideband capture:

$$E = \frac{B_{in} \cdot 2 \cdot \text{res}}{R_{out} \cdot \text{size}} \approx 10^7$$

Insight: *Autonomous signal intelligence is a problem of Information Theory: we must transform high-entropy raw IQ samples into low-entropy actionable tokens with zero loss of semantic intent.*

This represents a reduction of seven orders of magnitude, achieved through a three-stage pipeline of energy detection, ML-based classification, and LLM-driven narrative synthesis.

: Lossless Semantic Compression

While the pipeline reduces numerical data volume, it must maintain perfect semantic fidelity for 'Signals of Interest' (SOI). Metadata reduction is lossy by design; SOI extraction is lossless within the Nyquist bandwidth of the target emission.

6 The Semantic Horizon in RF Systems

In any sensor-driven intelligence platform, there exists a critical epistemological boundary where continuous physical phenomena (analog waveforms) must be collapsed into discrete, logical facts (a "Target"). We define this boundary as the **Semantic Horizon**.

Axiom 1 (The Semantic Horizon). *Below the horizon, data is probabilistic, continuous, and physical (represented as complex IQ samples and FFT bins). Above the horizon, data is deterministic, discrete, and logical (represented as JSON objects, schemas, and relational rows).*

Theorem 1 (Deterministic AI Constraint). *Machine Learning (CNNs, LLMs) must only be used as a function to cross the Semantic Horizon (classification) or to render post-horizon data (narrative RAG). AI must never be permitted to generate or infer structural facts that are not explicitly present in the post-horizon schema.*

Insight: *Spectra's primary architectural directive is to cross the Semantic Horizon as early as possible in the processing pipeline, minimizing the transmission of sub-horizon probabilistic data.*

7 Signal Detection

7.1 Problem Statement

Given a stream of FFT power spectra (one per frame, typically 10–30 fps), identify frequency regions that contain active transmissions and pass them downstream for classification.

7.2 Design Options Considered

A. Adaptive noise floor estimation Continuously estimate the noise floor using a sliding median and trigger on exceedance. Used by GNU Radio and many commercial receivers.

B. Fixed energy threshold Compare each FFT bin against a configurable power threshold in dB. Simple and deterministic.

C. Matched-filter detection Correlate against known signal templates. Optimal for known signals but requires a template library.

7.3 Decision and Rationale

Spectra uses **Option B (fixed energy threshold)** with configurable parameters. The rationale: simplicity and determinism. In a system that already has a downstream ML classifier, the detector's job is not to identify signals — only to find frequency regions worth classifying. A simple threshold with a merge step and minimum-bandwidth filter achieves this reliably. Adaptive noise-floor estimation adds complexity and state that is difficult to reason about during debugging.

7.4 Implementation

The `EnergyDetector` class implements a three-phase pipeline:

Algorithm 1 Energy-based signal detection

Require: Power spectrum $P[0..N-1]$ in dB, frequency axis $F[0..N-1]$
Require: Threshold τ (dB), merge gap g (Hz), min bandwidth $b_{\min}$ (Hz)
1: $above \leftarrow \{i : P[i] > \tau\}$
2: Group consecutive indices in $above$ into contiguous regions
3: **for** each region $[i_{\text{start}}, i_{\text{end}}]$ **do**
4: Create `DetectedSignal` with $F[i_{\text{start}}]$, $F[i_{\text{end}}]$, $\max(P[i_{\text{start}}..i_{\text{end}}])$, $\text{mean}(P[i_{\text{start}}..i_{\text{end}}])$
5: **end for**
6: Merge signals whose gap $< g$
7: Discard signals whose bandwidth $< b_{\min}$
8: **return** remaining signals

Default parameters: $\tau = -80$ dB, $g = 2000$ Hz, $b_{\min} = 5000$ Hz.

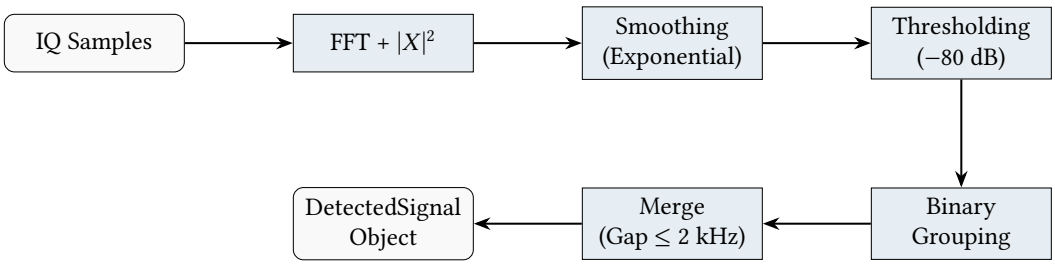

Figure 3. Signal detection pipeline. Raw samples are converted to power spectra, smoothed, and thresholded to produce discrete signal objects.

7.5 Consequences and Trade-offs

The fixed threshold means that in noisy environments, the detector may produce false positives (the classifier handles them). In quiet environments, weak signals below −80 dB are missed. Operators can adjust `threshold_db` per-device via the mission configuration.

8 Two-Stage Classification

8.1 Problem Statement

Given a detected signal (frequency bounds + IQ samples), assign a modulation label from a set of 14 standard classes: AM-DSB, FM, WBFM, BPSK, QPSK, 8PSK, QAM16, QAM64, GFSK, CPFSK, OOK, and others.

8.2 Design Options Considered

A. Single-pass classifier Run one model on every detection. Simple but slow models limit throughput; fast models sacrifice accuracy on edge cases.

B. Two-stage pipeline A fast model for real-time classification with a confidence gate. Low-confidence results are queued for retrospective deep analysis using a larger model and more IQ samples.

C. Ensemble voting Run multiple models and vote. Highest accuracy but highest cost.

8.3 Decision and Rationale

Spectra uses **Option B (two-stage pipeline)** because it gives the best latency/accuracy trade-off. Most signals (strong FM broadcasts, WBFM, etc.) are classified with high confidence in the first stage. Only ambiguous signals (weak digital modes, overlapping emissions) trigger the expensive retrospective path.

8.4 Implementation

Listing 6. Classification pipeline configuration

```
@dataclass
class PipelineConfig:
    confidence_threshold: float = 0.6   # Below this triggers retrospective
    retrospective_enabled: bool = True
    max_retrospective_queue: int = 32
    retrospective_sample_count: int = 8192
    detection_threshold_db: float = -80.0
    sigidwiki_enabled: bool = True
    sigidwiki_on_low_confidence: bool = True
```

The real-time stage runs on every FFT frame. When confidence falls below 0.6, the signal's frequency bounds and timestamp are placed on a bounded async queue. A background coroutine retrieves historical IQ samples from the `CircularIQBuffer` (a rolling window of recent frames) and runs the retrospective model.

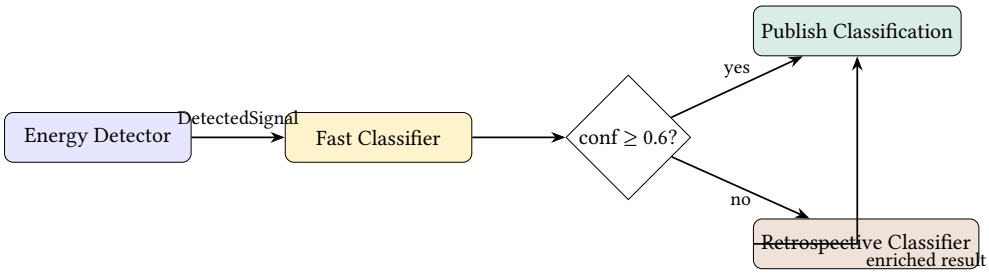

Figure 4. Two-stage classification pipeline. The confidence gate at 0.6 routes low-confidence detections to the retrospective classifier.

8.5 Model Backends

Three inference backends are supported, selectable at startup:

Backend	Runtime	Notes
CoreML	Apple Neural Engine	Fastest on Apple Silicon (M-series)
ONNX Runtime	CPU / GPU	Cross-platform, used for deployment
PyTorch	CPU / MPS	Used during training and development

: Supported Modulations

The primary classifier identifies 14 distinct modulation types, enabling broad spectrum monitoring across analog and digital domains:

Analog	Digital (ASK/PSK)	Digital (QAM/FSK)
AM-DSB	OOK	16QAM
AM-SSB	4ASK	64QAM
FM	BPSK	GFSK
WBFM	QPSK	CPFSK
AM-DSB-SC	8PSK	

8.6 SigIDWiki Enrichment

When the ML classifier produces a label, the `metadata/sigidwiki.py` module optionally queries the Signal Identification Wiki database to enrich the classification with known service names, frequency allocations, and protocol descriptions. This is especially useful for narrowband digital modes (POCSAG, DMR, P25) where the modulation label alone ("FSK") is not informative.

> **Warning**
>
> SigIDWiki enrichment requires network access and adds 50–200 ms latency. It is disabled by default in low-latency mission profiles and enabled only when `sigidwiki_on_low_confidence` is set.

9 Autonomous Missions

9.1 Problem Statement

An unattended monitoring station must scan designated frequency bands, log detections, and stop when a condition is met (time limit, hit count, rarity threshold) — all without operator intervention.

9.2 Design Options Considered

A. Cron-based sweeps Schedule frequency sweeps as periodic jobs. Simple but inflexible — cannot react to what is being received.

B. Mission engine with priority arbitration Define missions as first-class objects with lifecycle state, stop conditions, and priority-based hardware steering.

C. Rule-based reactive system Define triggers ("if signal X detected, then do Y"). Flexible but hard to reason about in combination.

9.3 Decision and Rationale

Spectra uses **Option B (mission engine)** because missions are the natural unit of operator intent. An operator says "watch 430–440 MHz for 30 minutes, capture anything unusual at FULL depth." This maps directly to a `MissionDef`.

The priority arbitration model allows concurrent missions without resource conflicts: only the highest-priority active mission steers the SDR hardware. Lower-priority missions continue to observe and log but cannot retune.

9.4 Implementation

The `MissionEngine` manages the full lifecycle:

1. **Definition**: operator creates a `MissionDef` (frequency range, dwell time, modulation filters, capture level, stop conditions).
2. **Scheduling**: the `MissionScheduler` queues the mission and transitions it to `running` when resources are available.
3. **Execution**: the engine retunes the SDR (if this mission holds steering priority), receives detections from the pipeline, scores each hit via the `NotabilityScorer`, and evaluates stop conditions.

4. **Reporting**: on completion, the `BriefingGenerator` produces a structured markdown briefing and daily logbook entry.
5. **Focus reload**: the `FocusStore` writes current scanning constraints to a file, polled every 500 ms. External tools can modify the focus file to steer the mission in real time.

9.5 Case Study: Identifying an Unknown UAS Link

To demonstrate the utility of autonomous missions, consider a scenario where an unknown Frequency Hopping Spread Spectrum (FHSS) signal appears in the 2.4 GHz ISM band. Traditional monitoring might log this as generic interference.

With Spectra, a `BandWatch` mission triggers a chain of autonomous actions:

1. **Detection**: The energy detector identifies 10 MHz bursts hopping across the band.
2. **Classification**: The ML classifier labels the modulation as `GMSK` with 94% confidence.
3. **Fingerprinting**: SEI features are extracted from the burst transients and compared against the database, matching a known `DJI OcuSync 2.0` controller fingerprint with a 0.92 cosine similarity score.
4. **Reporting**: The `BriefingGenerator` synthesizes these findings: *"Suspected DJI UAS activity detected at 14:02 UTC. High confidence match for OcuSync 2.0 protocol on 2412.5 MHz. Emitter ID matched to local drone registry (ID: DR-5512)."*

This entire chain occurs in under 1.5 seconds, transforming a meaningless energy burst into actionable intelligence.

9.6 Stop Conditions

Three stop-condition types are supported, composable via AND/OR:

Condition	Parameter	Semantics
Time limit	`max_duration_s`	Mission ends after N seconds
Hit count	`max_hits`	Mission ends after N detections
Rarity	`min_notability`	Mission ends when a signal with notability $\geq$ threshold is found

9.7 Narrated Briefing Generation

Upon mission completion, the `BriefingGenerator` acts as the final stage of the intelligence funnel. It aggregates the raw `MissionRun` data, including total detections, unique classifications, and extracted metadata.

Source

Instead of a static table, it uses a zero-shot LLM prompt to synthesize a *Narrated Briefing*—a concise, professional intelligence summary tailored for human operators. These briefings form the core of Spectra's "Daily Logbook" feature, reducing thousands of events into a paragraph of tactical insight.

9.8 Consequences and Trade-offs

The single-SDR steering model means that only one mission at a time controls the radio. In a multi-SDR deployment, each SDR can be assigned to a different mission via the `device_name` parameter. The focus-file polling mechanism (500 ms interval) introduces up to 500 ms of steering latency, acceptable for HF/VHF monitoring but potentially too slow for burst-mode signals.

10 Emitter Fingerprinting (SEI)

10.1 Problem Statement

Two transmitters on the same frequency using the same modulation appear identical to the classifier. Specific Emitter Identification (SEI) aims to distinguish individual radios by their hardware-specific RF signatures — imperfections in the oscillator, mixer, and ADC that produce unique “fingerprints” in the IQ data.

10.2 Design Options Considered

A. Handcrafted features + nearest-neighbour Extract known features (I/Q imbalance, CFO, transient shape) and classify with k-NN.

B. Deep metric learning (TripletNet) Train a neural network to map IQ bursts into an embedding space where same-emitter bursts cluster. Use vector search for identification.

C. Raw IQ classification Train a supervised classifier directly on emitter-labelled IQ data. Requires labelled data for every emitter.

10.3 Decision and Rationale

Spectra uses **Option A for feature extraction** combined with **Option B for identification**. Handcrafted features (I/Q imbalance, phase noise, transient edge shape via GLRT) are computed first, then concatenated with learned embeddings from the TripletNet. This hybrid approach gives interpretable features (useful for debugging) while benefiting from the representation power of deep learning.

10.4 Feature Extraction

The `fingerprinting/feature_extract.py` module computes:

- **I/Q imbalance**: amplitude ratio g and phase error ϕ, computed as:

$$g = \sqrt{\frac{E[Q^2]}{E[I^2]}}, \qquad \phi = \arcsin\left(\frac{E[IQ]}{\sqrt{E[I^2]}\,\sqrt{E[Q^2]}}\right)$$

  Perfect balance gives g=1, ϕ=0.
- **Carrier frequency offset (CFO)**: estimated from the phase rotation rate of the IQ stream.
- **Transient edge shape**: the GLRT (Generalised Likelihood Ratio Test) detector in `fingerprinting/transient_det.p` identifies burst edges. The transient envelope at turn-on is a hardware signature.

10.5 Vector Search

Fingerprint vectors are stored in ChromaDB. When a new burst arrives, its embedding is compared against the database using cosine similarity. If the nearest neighbour exceeds a configurable threshold, the burst is linked to that emitter's ID in the DuckDB `emitters` table.

Emitter linking flow

```
# 1. Extract features from IQ burst
g, phi = estimate_iq_imbalance(iq_data)
cfo = estimate_cfo(iq_data, sample_rate)

# 2. Compute TripletNet embedding
embedding = triplet_model.encode(iq_data)

# 3. Concatenate handcrafted + learned features
feature_vec = np.concatenate([[g, phi, cfo], embedding])

# 4. Search ChromaDB for nearest emitter
match = vector_search.query(feature_vec, threshold=0.85)

# 5. Link detection to emitter in DuckDB
if match:
    persistence.link_hit_to_emitter(hit_id, match.emitter_id)
```

11 Protocol Decoding

11.1 Problem Statement

Once a signal is classified (e.g., as "POCSAG" or "DMR"), the raw IQ data should be decoded into human-readable protocol content — pager messages, radio IDs, RDS station names.

11.2 Design Options Considered

A. Native decoders in Python Write decoders from scratch. Full control but enormous effort for mature protocols.

B. Subprocess wrappers around established tools Shell out to Multimon-ng, DSDcc, and similar tools. Leverages decades of community work.

C. Rust decoder library Compile decoders to a shared library callable from Python. Best performance but highest integration cost.

11.3 Decision and Rationale

Spectra uses **Option B (subprocess wrappers)** because Multimon-ng and DSDcc are battle-tested decoders with wide protocol coverage. The wrapper approach means Spectra benefits from upstream bugfixes without maintaining its own codec implementations.

11.4 Implementation

The `ProtocolManager` in `decoders/protocol_manager.py` manages decoder subprocess lifecycles:

- **Multimon-ng**: decodes POCSAG, RDS, DTMF. Receives audio via stdin pipe; emits JSON on stdout.
- **DSDcc**: decodes DMR, P25. Receives audio via stdin; emits voice metadata extracted by regex from stderr.

Decoded content is stored in the DuckDB `decoded_content` table, linked to the originating signal hit by `hit_id`.

> **Warning**
>
> Subprocess-based decoding adds 100–300 ms startup latency per decoder instance. The `ProtocolManager` keeps decoder processes alive across detections to amortise this cost. If a decoder crashes, it is automatically restarted on the next matching detection.

12 Constellation Analysis & Timing Recovery

While modulation classification provides a label, manual verification of digital signals often requires visualising the symbol constellation. Spectra provides an automated constellation extractor that recovers symbol timing from oversampled IQ streams.

12.1 Mueller-Muller Clock Recovery

To extract symbol-spaced points from an arbitrary IQ capture, Spectra employs a Mueller-Muller timing error detector (TED). The process follows three stages:

Source

1. **Interpolation**: The IQ stream is upsampled by a factor of 16 using a polyphase FIR filter to allow for fractional timing adjustments.
2. **Error Detection**: A Mueller-Muller TED calculates the timing offset τ based on the transition between the current symbol y_k and the previous symbol y_{k-1} relative to their hard-decision "rail" values.
3. **Loop Filter**: A first-order loop filter adjusts the sampling phase μ based on the error, walking forward through the interpolated stream to pick the optimal symbol sample.

12.2 Metric Extraction

Once symbols are recovered, Spectra calculates:

- **EVM (Error Vector Magnitude)**: The RMS distance between recovered symbols and their ideal constellation points.
- **Cluster Count**: Estimated using a k-means clustering heuristic to confirm if a signal is e.g., 4-PSK vs 16-QAM.

13 Autonomous Satellite Intelligence

Fixed-frequency monitoring is insufficient for Low Earth Orbit (LEO) satellites, which are only visible during brief "passes." Spectra includes a TLE-based scheduler for autonomous satellite acquisition.

13.1 TLE Propagation

Spectra uses the `skyfield` library to propagate Two-Line Element (TLE) sets for a configured list of satellites (e.g., NOAA weather, Orbcomm, CubeSats). By combining TLE data with the observer's GPS coordinates, the system predicts:

Source

- **AOS/LOS**: Acquisition and Loss of Signal times.
- **Doppler Shift**: Expected frequency offset due to relative velocity (up to ±50 kHz at 137 MHz).
- **Maximum Elevation**: Used to prioritize passes with better link margins.

13.2 Autonomous Mission Scheduling

The `SatelliteScheduler` generates `PassWindow` objects. When a pass begins, the `MissionEngine` automatically:

1. Tunes the SDR to the satellite's downlink frequency.
2. Applies a real-time Doppler correction shift.
3. Triggers a `DEEP` capture to record the full bandwidth of the pass for retrospective decoding.

14 Autopilot Sweep & Frequency Allocation

Beyond directed missions, Spectra features an autonomous "Autopilot" sweep mode that continuously explores the RF spectrum to build a pattern of life.

14.1 Bandplan Allocation

The frequency domain is divided into logical segments defined by a JSON `bandplan`. Each `BandSegment` specifies:

Source

- **Start/End Frequencies**: The bounds of the segment (e.g., VHF Marine).
- **Step Size**: The tuning increment.
- **Dwell Time**: How long the SDR should monitor the segment before moving on (e.g., 2.0 seconds).

14.2 Autopilot Sweep Prioritisation

To avoid wasting time on empty bands, the `InterestingnessScorer` uses a deterministic heuristic to score each segment dynamically. Scoring is based on local DuckDB hit history:

Source

- **Novelty**: Fewer hits in the last 7 days increases the score, encouraging exploration of unknown bands.
- **Activity**: More hits in the last hour increases the score, focusing attention on active events.
- **Anomaly**: Deviations from baseline bandwidth or label distributions trigger a higher score.

The autopilot scheduler then prioritises segments with the highest "interestingness" score, creating a self-optimizing sweep pattern.

15 Data Capture & SigMF Recording

While metadata extraction represents the core of Spectra's deterministic entropy reduction, raw signal capture is often required for deep-dive forensics, model training, and post-event analysis.

15.1 SigMF Standard Compliance

Spectra writes raw IQ data using the Signal Metadata Format (SigMF v1.2.6) standard. Each recording generates two files:

Source

- `.sigmf-data`: The raw interleaved complex float32 little-endian IQ samples (`cf32_le`).
- `.sigmf-meta`: A JSON file containing the sample rate, center frequency, ISO 8601 timestamp, and datatype annotations.

15.2 Circular Buffering

To support "retrospective" capture (e.g., saving the 5 seconds *before* a signal was identified), Spectra maintains a `CircularIQBuffer`. When a capture mission triggers, the buffer's contents are seamlessly prepended to the live stream before flushing to disk as a complete SigMF archive.

16 Synthetic RF Training Lab

16.1 Problem Statement

Training a modulation classifier requires large, labelled IQ datasets. Over-the-air recordings are expensive to collect and difficult to label accurately. Synthetic data generation provides unlimited, perfectly labelled training examples, but the quality of the classifier depends on the realism of the synthetic data.

16.2 Design Options Considered

A. Use existing datasets (RadioML, DeepSig) Pre-made datasets. Convenient but limited to fixed SNR ranges and modulation sets.

B. Generate synthetic baseband + channel impairments Write deterministic baseband generators for each modulation, then apply configurable channel models (AWGN, Doppler, Rayleigh fading).

C. GAN-based augmentation Train a GAN to generate realistic IQ samples. High realism but difficult to control and label.

16.3 Decision and Rationale

Spectra uses **Option B (deterministic generators + channel models)** because it gives complete control over the signal-to-noise ratio, fading profile, and frequency offset of each training example. Labels are ground-truth by construction. The synthetic pipeline can generate arbitrarily large datasets in minutes on a single CPU.

16.4 Implementation

The synthetic lab comprises five modules:

Module	Responsibility
`synthetic/generator.py`	Baseband IQ: AM, FM, FSK, BPSK, QPSK, QAM
`synthetic/channel.py`	Channel impairments: AWGN, Doppler, Rayleigh
`synthetic/dataset.py`	PyTorch Dataset for SigMF recordings
`synthetic/model.py`	1D-ResNet classifier architecture
`synthetic/train.py`	Training loop; exports ONNX

Generating a synthetic QPSK signal with channel impairments

```
from spectra.intelligence.synthetic.generator import generate_qpsk
from spectra.intelligence.synthetic.channel import apply_awgn, apply_doppler

# Deterministic baseband generation
rng = np.random.default_rng(seed=42)
iq = generate_qpsk(fs_hz=2e6, duration_s=0.01,
                   carrier_hz=0, sym_rate=50e3, rng=rng)

# Channel impairments
iq = apply_awgn(iq, snr_db=10.0, rng=rng)
iq = apply_doppler(iq, fs_hz=2e6, shift_hz=150.0)
# iq is now a labelled "QPSK at 10 dB SNR" training example
```

The training loop in `synthetic/train.py` fine-tunes a 1D-ResNet on the generated data and exports the result as an ONNX model, ready for deployment via the ONNX Runtime backend.

17 Audio Content Intelligence

17.1 Problem Statement

Voice-bearing signals (FM broadcast, AM aviation, SSB amateur) carry semantic content that cannot be captured by modulation classification alone. Converting demodulated audio to text enables full-text search over the signal census and richer briefings.

17.2 Design Options Considered

A. Cloud STT (Whisper API, Google STT) Highest accuracy but requires internet access and introduces latency and privacy concerns.

B. Local Whisper via mlx-whisper Runs entirely on Apple Silicon using the MLX framework. Moderate accuracy, zero network dependency.

C. Vosk / PocketSphinx Lightweight local inference. Lower accuracy than Whisper.

17.3 Decision and Rationale

Spectra uses **Option B (mlx-whisper)** because the target deployment platform is Mac mini M-series, which has dedicated Neural Engine hardware that MLX exploits. Running locally means no network dependency, no per-minute cost, and no privacy risk from sending intercepted audio to a cloud service.

17.4 Implementation

The audio intelligence pipeline has three stages:

1. **Demodulation**: the `AudioStreamer` translates the SDR's IQ stream into the signal of interest. It frequency-shifts the target signal to baseband, applies a channel filter, decimates, and demodulates using the appropriate demodulator (FM/AM/SSB). Output is 48 kHz float audio.
2. **Voice Activity Detection (VAD)**: the audio streamer accumulates 5-second chunks. When energy exceeds a threshold (indicating voice rather than silence or noise), the chunk is dispatched for transcription.
3. **Transcription**: the `AudioTranscriber` resamples to 16 kHz (Whisper's required rate using `scipy.signal.resample_poly`), runs mlx-whisper inference, and returns the text. Transcripts are stored in DuckDB alongside the originating signal hit.

Listing 7. AudioTranscriber core

```
class AudioTranscriber:
    def __init__(self, model_path="mlx-community/whisper-tiny-mlx"):
        self.model_path = model_path
        self.target_fs = 16000  # Whisper requires 16 kHz

    def transcribe(self, audio_data: np.ndarray, sample_rate: int) -> str:
        audio_16k = self._resample_if_needed(audio_data, sample_rate)
        result = mlx_whisper.transcribe(
            audio_16k, path_or_hf_repo=self.model_path, fp16=True
        )
        return result.get("text", "").strip()
```

> **Note**
>
> The Whisper "tiny" model is used by default for speed (<1 s per 5-second chunk on M2). For higher accuracy on noisy RF audio, operators can switch to "small" or "medium" by changing `model_path`.

18 Natural Language Querying (RAG)

To make the gigabytes of historical signal data accessible to non-technical users, Spectra implements a Natural Language Query (NLQ) interface using Retrieval-Augmented Generation (RAG).

18.1 SQL Synthesis Pipeline

Source

Unlike traditional RAG which retrieves text chunks, Spectra's NLQ engine converts natural language into structured DuckDB SQL queries. The process follows a two-step LLM chain:

1. **SQL Generation**: The system provides the LLM (e.g., Llama 3) with the DuckDB schema and a set of "Critical Rules" for frequency filtering and table joining. The LLM produces a single SQL query.
2. **Data Synthesis**: The SQL query is executed against the local DuckDB signal history. The raw result set is then passed back to the LLM with a synthesis prompt to generate a professional intelligence briefing.

: NLQ Critical Rules

To ensure accurate SQL generation, the LLM is constrained by a system prompt that enforces:

- **Frequency Normalization**: Queries for "156.8 MHz" are automatically expanded to range checks (`BETWEEN 156.7e6 AND 156.9e6`).
- **Payload Extraction**: Transcript text must be extracted from JSON payloads using `json_extract_string(payload, '$.text')`.
- **Temporal Joins**: Protocol decodes (`decoded_content`) must be joined with signal hits (`signal_hits`) to recover the signal's timestamp and frequency context.

18.2 Example Briefing Output

A query like *"Show me all voice transcripts from the last hour near 156.8 MHz"* results in:

> *"Between 13:00 and 14:00 UTC, four voice transmissions were intercepted on the Marine VHF Distress channel (156.8 MHz). Transcripts indicate routine 'Securite' navigational warnings from the Coast Guard regarding a buoy displacement in sector Alpha."*

19 SDR Hardware Server (`rtltcp-rust`)

19.1 Problem Statement

Different SDR hardware (RTL-SDR, AirSpy R2, AirSpy HF+, Kraken 5-channel coherent) each have their own driver APIs. Clients should not need to know which hardware is connected.

19.2 Design and Implementation

`rtltcp-rust` is a standalone Rust server that presents a unified interface:

- **TCP IQ streaming**: standard RTL-TCP protocol for backward compatibility with existing tools (e.g., SDR#, GQRX), plus an extended RTE1 protocol with a 5-byte frame header (1 byte type + 4 bytes LE length) for future extensibility.
- **REST API**: device enumeration, health checks, and system metrics.

Endpoint	Method	Description
`/api/v1/health`	GET	Server status
`/api/v1/server`	GET	Name, version, uptime, device count
`/api/v1/devices`	GET	List SDRs with stats
`/api/v1/devices/{name}`	GET	Single device detail
`/api/v1/system`	GET	CPU, memory, temperature

The server uses Tokio for async I/O. Each connected client gets a dedicated task that reads from the device's sample ring buffer, applies any per-client frequency/gain settings, and writes IQ frames to the TCP socket.

RTE1 protocol

The 5-byte RTE1 header was chosen over the standard 4-byte RTL-TCP header to allow a type byte for future message types (metadata, calibration data, device events) without breaking

> backward compatibility. Clients that send the standard RTL-TCP handshake receive standard framing; clients that send the RTE1 handshake receive extended framing.

19.3 Zero-Conf Discovery (mDNS)

To support headless edge deployments where IP addresses are dynamically assigned, Spectra utilizes Multicast DNS (mDNS). The `rtltcp-rust` server broadcasts its availability over the local network via Avahi/Bonjour.

Source

When the Spectra backend starts, the `ServerDiscovery` module listens for `_rtltcp._tcp.local` services. Upon discovery, it automatically resolves the IP and port, negating the need for manual configuration files.

19.4 Operational Health Monitoring

Distributed DSP systems are prone to silent failures like USB bus dropouts or thermal throttling. The `HealthMonitor` subsystem continuously polls the SDR server and the local inference engines to track:

Source

- **Buffer Overflows**: Indicates if the CPU cannot keep up with the IQ stream.
- **Device Temperature**: Warns if the SDR is overheating.
- **Inference Latency**: Tracks the rolling average time per ML classification.

These metrics are exposed via the WebSocket `META` channel for client dashboards.

20 Rust TUI (`spectra-tui`)

The terminal UI operates in two modes:

Basic mode Direct TCP connection to `rtltcp-rust`. Displays waterfall and spectrum. No classification or audio. Suitable for lightweight monitoring when the Python backend is not running.

Advanced mode WebSocket connection to the Python backend. Receives classified detections, audio, and briefings via the TLV protocol. Full feature set.

20.1 GPU-Accelerated Waterfall

The optional `native-waterfall` feature compiles a WGPU compute shader that renders the waterfall image on the GPU. The CPU-side ring buffer pushes new spectral rows; the shader maps power values to a colour palette and composites the scrolling image. This achieves smooth rendering at 60+ fps even with 4096-bin FFTs.

When WGPU is not available (e.g., over SSH), the TUI falls back to a CPU-based renderer using Kitty graphics protocol for inline image display.

20.2 Dual-Mode Architecture Rationale

Keeping basic mode ensures the TUI is useful even without the full Python stack. A field operator can connect a single RTL-SDR, launch the TUI, and get a waterfall display with no dependencies beyond the Rust binary and `rtltcp-rust`. Advanced mode adds intelligence when the infrastructure is available.

21 Frontend Web UI Architecture

While the Rust TUI provides low-latency tactical monitoring, the React-based Web UI (`spectra-web`) serves as the primary intelligence dashboard for historical analysis and distributed monitoring.

21.1 Component Architecture

The frontend is built using React and TypeScript, structured around several core views:

Source

- **TacticalMap**: A 3D geospatial view using CesiumJS to visualise distributed edge nodes and Time Difference of Arrival (TDOA) positioning vectors.
- **DeepCensus**: A hierarchical browser for historical signals, grouping detections by frequency band, classification, and emitter fingerprint.

- **AskView**: The conversational interface for the RAG/NLQ engine, supporting streaming markdown responses.
- **Waterfall & Constellation**: Real-time visualization components driven by the WebSocket TLV stream using HTML5 Canvas.

21.2 State Management & Hooks

The UI avoids heavy global state managers (like Redux), preferring custom React hooks (`useSpectraSocket`, `useMissions`) to encapsulate the WebSocket connection lifecycle, binary TLV parsing, and automatic reconnection logic.

Source

22 Hardware Optimization & SIMD Acceleration

The real-time constraints of spectrum monitoring require efficient computation, especially for the high-bandwidth Fast Fourier Transform (FFT) and FIR filtering stages. Spectra leverages platform-specific hardware acceleration to minimize CPU usage.

22.1 Apple Silicon vDSP Acceleration

On macOS systems running Apple Silicon (M1/M2/M3), Spectra utilizes the `Accelerate.framework` via a `ctypes` binding to the `vDSP` library. This provides significant performance benefits over standard NumPy/FFTW:

Source

- **Split-Complex Memory Layout**: `vDSP_fft_zip` uses a split-complex layout (separate arrays for real and imaginary parts), optimizing memory bandwidth and cache usage.
- **SIMD Utilization**: vDSP functions are handwritten in assembly to leverage the ARM NEON SIMD instructions, allowing for the simultaneous processing of multiple signal samples.
- **Reduced Power Consumption**: Offloading FFTs to the Apple Silicon AMX (Apple Matrix Coprocessor) or dedicated DSP cores reduces thermal throttling and extends battery life on portable deployments.

22.2 Fallback Mechanism

Spectra detects the presence of the `Accelerate` framework at runtime. If unavailable (e.g., on Linux or non-Apple hardware), the system automatically falls back to `numpy.fft`, which typically uses MKL or OpenBLAS for optimization.

23 Edge Architecture

For distributed deployments, Spectra supports a hub-and-spoke model:

- **Edge nodes** (Raspberry Pi + RTL-SDR) run a lightweight Python agent that streams FFT spectra and sparse IQ to the central server via the `SPARSE_IQ` and `SPECTRUM` TLV channels.
- **Central server** (Mac mini M-series) runs the full Python backend, performing classification, fingerprinting, and persistence.

The edge ingest server (`core/edge_ingest_server.py`) accepts registrations from edge nodes, manages their lifecycles, and can issue demand-control messages (e.g., "send full IQ for the next 5 seconds") via the `DEMAND_CONTROL` channel.

> **Federation**
>
> For multi-site deployments, Spectra supports federated census via RQLite (`intelligence/federation/rqlite_bridge.py`). Each site maintains a local DuckDB census; the RQLite bridge replicates summaries to a shared distributed SQLite database for cross-site querying.

24 Insights and Evaluation

The efficiency of the signal intelligence pipeline is measured by its ability to process wideband IQ data in real-time.

Insight:
By offloading energy detection to the edge, we reduce the data volume by over 99% before ML classification occurs.

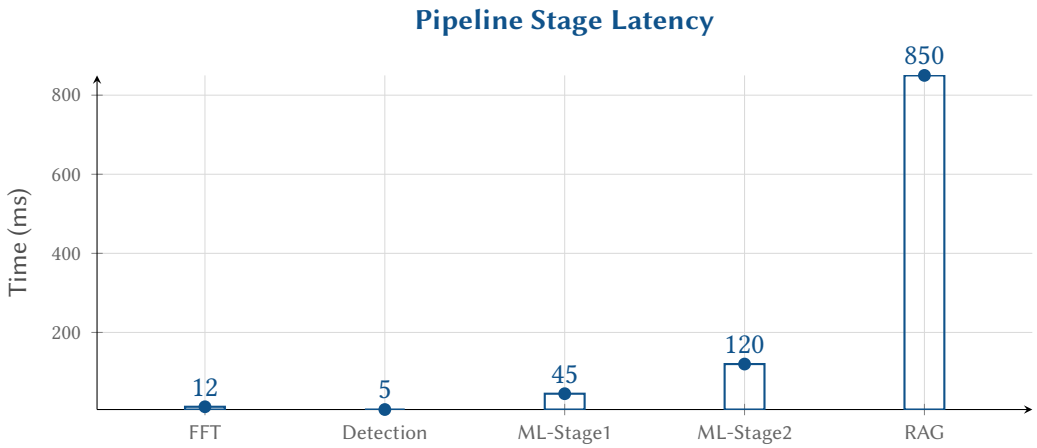

Figure 5. Spectra Pipeline Latency: RAG-based synthesis is the primary bottleneck.

: Metadata-First Transport

The backhaul between edge sensors and the central orchestrator must prioritize low-bandwidth metadata (detections, features) over raw IQ samples. SOI captures should only be triggered by autonomous mission logic or explicit operator override.

25 API Reference

25.1 REST Endpoints

Endpoint	Method	Description
`/api/health/deep`	GET	System health: device stats, WS queue pressure, audio counters
`/api/census/deep`	GET	Query signal census with time/frequency filters
`/api/archive/stats`	GET	Rarity index of captured signal types
`/api/emitters`	GET	List known emitters with fingerprint metadata
`/api/briefing`	GET	Narrated intelligence summary for time window
`/api/nlq`	POST	Natural-language query (SSE streaming response)
`/api/tdoa`	POST	Time Difference of Arrival geolocation
`/api/edge/register`	POST	Edge node registration
`/api/edge/sparse-ingest`	POST	Sparse IQ upload from edge nodes

25.2 WebSocket Endpoint

Endpoint	Description
`/ws/stream`	Multiplexed TLV stream. Clients subscribe to channels (SPECTRUM, DETECTIONS, CLASSIFICATION, AUDIO, etc.) via `SUBSCRIBE` messages. Server pushes data on subscribed channels.

25.3 WebSocket Channel Reference

ID	Channel	Payload format
0x01	SPECTRUM	Binary: 16-byte header (center_freq i64 + sample_rate i64) + u8 power bins
0x02	DETECTIONS	JSON: array of `DetectedSignal` objects
0x03	CLASSIFICATION	JSON: `SignalClassification` object
0x04	AUDIO	Binary: 8/24-byte header + i16 LE PCM samples
0x05	CONTROL	JSON: command/response
0x06	META	JSON: session metadata
0x07	SPARSE_IQ	Binary: complex64 IQ samples (edge ingest)
0x08	DEMAND_CONTROL	JSON: server-to-edge demand toggles

26 Advanced Spatial Intelligence & BSS

With the integration of v8.0 features, Spectra provides advanced spatial awareness of the RF environment.

26.1 Distributed Blind Signal Separation (BSS)

Spectra employs independent component analysis (FastICA) and federated beamforming to separate overlapping emissions. Overlapping signals, which traditionally confuse basic classifiers, are isolated into distinct streams, enabling clean classification and decoding even in highly congested bands.

26.2 3D Battlespace Visualization

The web interface (built with React and CesiumJS) overlays intercepted emitters onto a 3D topological map. This provides operators with a comprehensive spatial representation of the RF environment, integrating Time Difference of Arrival (TDOA) heatmaps and directional vectors in real-time.

27 Cognitive Radios & Resilient Communication

Spectra extends beyond monitoring into interactive and resilient RF operations.

27.1 Dynamic Spectrum Access (DSA)

By leveraging the comprehensive signal census, Spectra identifies underutilized spectrum in real-time. This capability supports intelligent frequency hopping and DSA, ensuring uninterrupted operations in contested environments.

27.2 Listen-Before-Talk (LBT)

The continuous spectrum monitoring pipeline naturally enables autonomous LBT strategies. Transmit operations can dynamically adapt to the presence of primary users or jamming signals, mitigating interference and avoiding detection.

28 The Exploration Frontier

Version 9.0 introduces capabilities aimed at uncharacterized signals and environmental anomalies.

28.1 Unsupervised Protocol Discovery

Spectra utilizes clustering algorithms on raw IQ and demodulated feature spaces to identify completely novel, uncharacterized protocols. This breaks the reliance on known signatures and allows the platform to flag anomalous communications for further analysis.

28.2 Meteor Scatter Astronomy & Anomalous Propagation

The platform incorporates physics-based propagation models. By continuously logging signal intensities over long distances, Spectra can identify anomalous propagation events such as meteor scatter, sporadic E-layer reflections, and tropospheric ducting.

29 Limitations

29.1 Single-SDR Steering

Description: Only one mission at a time can steer each SDR. Concurrent missions that require different frequencies on the same device cannot both observe their target bands simultaneously.

Workaround: Deploy multiple SDRs and assign each mission a dedicated `device_name`.

Planned resolution: Wideband SDRs (AirSpy HF+ at 768 kHz bandwidth) can cover multiple signals within a single tuning. Future work will add channelised observation where the SDR stays on a wide band and individual missions filter sub-bands.

29.2 Classifier Coverage

Description: The modulation classifier supports 14 standard classes. Exotic or proprietary modulations (e.g., DMR Tier III trunking, custom OFDM) are classified as the nearest standard modulation, often with low confidence.

Workaround: Use the synthetic training lab to generate training data for new modulation types and retrain the model.

Planned resolution: An "unknown" class with anomaly detection is under investigation.

29.3 NL Query Accuracy

Description: The text-to-SQL RAG pipeline uses a 1B-parameter local LLM. Complex queries (joins across multiple tables, time-range arithmetic) fail approximately 20% of the time.

Workaround: Rephrase the question in simpler terms, or use the `/api/census/deep` REST endpoint with explicit filters.

Planned resolution: Upgrade to a larger local model (7B+) when Apple Silicon memory permits, or add a query-validation step.

29.4 Whisper Accuracy on Noisy RF Audio

Description: mlx-whisper "tiny" struggles with weak, faded, or heavily-interfered voice signals. Transcription error rates on aviation AM (high noise, clipped audio) can exceed 30%.

Workaround: Switch to the "small" or "medium" Whisper model for critical monitoring tasks.

Planned resolution: Fine-tune Whisper on RF-degraded audio samples.

29.5 Edge Node Latency

Description: Edge nodes (Raspberry Pi) introduce 200–500 ms of additional latency for spectrum data due to network transport and the sparse IQ batching interval.

Workaround: Run the full backend on the edge node if the hardware supports it (Pi 5 with 8 GB RAM).

Planned resolution: No current plan. The latency is acceptable for the target use case (unattended monitoring, not real-time interception).

30 Future Work & Experimental Features

While the core Spectra pipeline is production-ready, several advanced DSP and machine learning features are currently in the experimental phase (`src/spectra/intelligence/experimental/`).

30.1 Neural RF Denoising

Weak signals (SNRs below 5 dB) often fail the classification gate, falling back to generic "energy burst" detections. We are developing a 1D U-Net autoencoder designed to operate directly on the raw complex IQ stream. Trained on the synthetic RF dataset (which provides perfect ground-truth

clean signals paired with AWGN-corrupted variants), the denoiser aims to reconstruct the clean baseband signal *before* it reaches the classifier, effectively lowering the minimum discernible signal (MDS) floor.

30.2 Distributed Blind Signal Separation (BSS)

Source

In congested bands (like 433 MHz ISM), multiple independent emitters often transmit simultaneously, causing collisions that confound standard classifiers. The experimental separation module uses FastICA (Independent Component Analysis) applied across multiple spatially distributed edge sensors. By treating each SDR as an antenna in a widely dispersed array, Spectra can mathematically un-mix the overlapping signals into distinct IQ streams for independent classification and decoding.

30.3 Other Planned Enhancements

- **Wideband channeliser**: simultaneous observation of multiple sub-bands within a single SDR tuning, enabling concurrent missions.
- **Anomaly detection**: an "unknown modulation" class that flags signals not matching any known pattern.
- **ONNX-on-device for edge**: run the modulation classifier directly on Raspberry Pi, reducing central server load.

A DuckDB Schema Reference

Listing 8. Complete Deep Intelligence schema

```
-- Signal hits: every persisted detection
CREATE TABLE signal_hits (
    id              VARCHAR PRIMARY KEY,
    timestamp_ms    BIGINT NOT NULL,
    center_freq_hz  DOUBLE NOT NULL,
    sample_rate_hz  DOUBLE NOT NULL,
    freq_start_hz   DOUBLE NOT NULL,
    freq_end_hz     DOUBLE NOT NULL,
    bandwidth_hz    DOUBLE NOT NULL,
    peak_db         DOUBLE NOT NULL,
    mean_power_db   DOUBLE NOT NULL,
    device_name     VARCHAR NOT NULL,
    emitter_id      VARCHAR,
    label           VARCHAR,
    confidence      DOUBLE,
    artifact_path   VARCHAR
);

-- Emitters: unique hardware fingerprints
CREATE TABLE emitters (
    id                  VARCHAR PRIMARY KEY,
    fingerprint_vector  FLOAT[],
    label               VARCHAR,
    last_seen           TIMESTAMP
);

-- Decoded content: protocol payloads linked to hits
CREATE TABLE decoded_content (
    id       VARCHAR PRIMARY KEY,
    hit_id   VARCHAR NOT NULL,
    protocol VARCHAR NOT NULL,
    payload  JSON
);
```

B TLV Protocol Quick Reference

Byte offset	Size	Field	Description
0	1	`channel`	Channel type (see §25)
1	1	`flags`	Bit 0: COMPRESSED; bits 1–7 reserved
2	2	`seq`	Sequence number (LE u16), wraps at 65535
4	4	`length`	Payload length in bytes (LE u32)
8	N	`payload`	Channel-specific data

B.1 Spectrum Payload Header (16 bytes)

Byte offset	Size	Field	Description
0	8	`center_freq`	Centre frequency in Hz (LE i64)
8	8	`sample_rate`	Sample rate in Hz (LE i64)
16	$N-16$	`bins`	Power values as u8 (0–255 mapped to dB range)

C API Endpoint Quick Reference

Endpoint	Method	Auth	Returns
`/api/health/deep`	GET	None	JSON: system health
`/api/census/deep`	GET	None	JSON: signal hit array
`/api/archive/stats`	GET	None	JSON: modulation rarity index
`/api/emitters`	GET	None	JSON: emitter array
`/api/briefing`	GET	None	JSON: narrated summary
`/api/nlq`	POST	None	SSE: streaming prose answer
`/api/tdoa`	POST	None	JSON: geolocation result
`/api/edge/register`	POST	None	JSON: registration ack
`/api/edge/sparse-ingest`	POST	None	JSON: ingest ack
`/ws/stream`	WS	None	TLV binary frames